Encapsulamiento en Taller ICNO

Proyecto Final de Programación Orientada a Objetos · Isaac Navarro · Código en **inc/taller.php**

1. Qué es el encapsulamiento y por qué lo usamos

Encapsular es **guardar los datos de un objeto bajo llave** y dejar que solo se lean o cambien por métodos que la propia clase ofrece. Los atributos se declaran `private`, así que ninguna página del sistema puede hacer `$servicio->costoBase = -5;`: PHP lo rechaza con un error. Para leer se usa un **getter** (`getCostoBase()`) y para cambiar un **setter** (`setCostoBase()`). La ventaja es que el objeto siempre controla su propio estado: si mañana hay que validar un dato, se hace en un solo lugar y todo el sistema queda protegido.

2. Cómo está aplicado: las tres reglas que cumplen las once clases

Regla	Cómo se ve en el código
a) Todos los atributos son `private`	Las 11 clases declaran sus atributos con `private $nombre;`. No hay ningún atributo `public` en el proyecto.
b) Acceso solo por getters y setters	Cada atributo que se necesita afuera tiene su `getX()`; los que pueden cambiar tienen además su `setX()`.
c) Se usa `$this` dentro de la clase	Constructores, getters y setters escriben `$this->atributo`, es decir, "el dato de *este* objeto", no de otro.

Ejemplo completo: la clase `Servicio` (líneas 28 a 54)

```php
class Servicio {
    private $id;            // Nadie fuera de la clase puede tocarlo directo.
    private $descripcion;
    private $costoBase;

    public function __construct($id, $descripcion, $costoBase) {
        $this->id = $id;                    // $this = "este objeto en particular".
        $this->descripcion = $descripcion;
        $this->costoBase = $costoBase;
    }
    public function getCostoBase() { return $this->costoBase; }             // Getter: lectura controlada.
    public function setCostoBase($costoBase) { $this->costoBase = $costoBase; } // Setter: cambio controlado.
    public function getDescripcion() { return $this->descripcion; }
    public function setDescripcion($d) { $this->descripcion = $d; }
    public function getId() { return $this->id; }
    public function setId($id) { $this->id = $id; }
}
```

El mismo patrón en el resto de las clases

Clase	Atributos privados	Getters / setters
Servicio	id, descripcion, costoBase	getter y setter de los 3
ServicioPreventivo	kilometraje	getKilometraje / setKilometraje
ServicioCorrectivo	horasManoObra	getHorasManoObra / setHorasManoObra
Vehiculo	id, placas, marca, modelo, color, anio, kilometraje	getter y setter de los 7 (líneas 118 a 131)
Auto, Camioneta, Camion, Motocicleta	ninguno propio; heredan los 7 de Vehiculo	heredan los getters y setters del padre
Usuario	id, nombre, correo, rol	solo getters (ver sección 3)
Cotizacion	id, vehiculo, servicios, total, estado, fecha	solo getters (ver sección 3)
Taller	bd (la conexión PDO)	ninguno: la conexión nunca sale de la clase

Las hijas también respetan lo privado del padre

Como `$costoBase` es `private` en `Servicio`, ni siquiera sus hijas lo ven directo. Por eso `ServicioPreventivo` calcula su costo con el getter del padre y no con el atributo:

```
#[\Override] public function calcularCosto() { return $this->getCostoBase() * 0.9; }   // linea 66
```

Y en el constructor de la hija, lo del padre lo guarda el padre con `parent::__construct()`; la hija solo guarda lo suyo (`$this->kilometraje`). Cada clase es dueña únicamente de sus propios datos.

3. Decisiones de diseño: no todo tiene setter a propósito

El encapsulamiento no es solo poner `private` y luego regalar un setter para todo. También es decidir **qué se puede cambiar y qué no**:

- **Usuario** tiene getters pero ningún setter. Un usuario no cambia de id, correo ni rol desde las páginas; eso solo lo hace la base de datos. Así nadie puede "ascenderse" a admin escribiendo `$u->setRol('admin')`.
- **Cotizacion** tampoco tiene setters. Una vez calculada es un comprobante: su total y sus servicios no se editan. Para marcarla como revisada se usa `Taller->marcarRevisada()`, que actualiza la base de datos.
- **Taller** guarda la conexión `$bd` privada y **sin getter**. Ninguna página puede obtener la conexión ni escribir SQL por su cuenta. Es la forma más fuerte de encapsular: el dato existe, pero solo la clase lo usa.
- Los métodos `usuarioDesdeFila()`, `vehiculoDesdeFila()`, `servicioDesdeFila()` y `cotizacionDesdeFila()` son `private`. Convertir una fila SQL en objeto es un detalle interno de Taller; las páginas solo reciben objetos ya armados.

4. Cómo trabajan las páginas con los objetos

Las páginas PHP (`servicios.php`, `cotizar.php`, `mis_cotizaciones.php`, etc.) nunca ven atributos ni tablas. Solo llaman métodos públicos. Ejemplos reales:

```
// servicios.php, lineas 157 a 165: la tabla del CRUD se dibuja con getters.
<td><?php echo $s->getId(); ?></td>
<td><?php echo $s->getTipo(); ?></td>
<td><?php echo htmlspecialchars($s->getDescripcion()); ?></td>
<td>$<?php echo number_format($s->getCostoBase(), 2); ?></td>

// cotizar.php, linea 34: el subtotal sale de dos metodos, sin tocar un solo atributo.
$sub = $s->calcularCosto() * $vehiculo->getFactor();

// mis_cotizaciones.php, linea 24: la cotizacion entrega su vehiculo como objeto, y ese objeto entrega sus datos.
$v = $c->getVehiculo();   echo $v->getMarca() . ' ' . $v->getModelo();
```

El flujo completo de un dato, de la base al navegador

#	Paso	Quién lo hace y con qué
1	La página pide el catálogo	`$taller->getServicios()` (método público de Taller)
2	Taller consulta MySQL	Usa su `$this->bd` privado con `prepare()` y parámetros `?`
3	Cada fila se vuelve objeto	`servicioDesdeFila()` (privado) crea un ServicioPreventivo o ServicioCorrectivo con `new`; el constructor llena los atributos privados con `$this`
4	La página recibe una lista de objetos	Solo puede usar getters: `getDescripcion()`, `getCostoBase()`, `calcularCosto()`
5	El admin edita un servicio	El formulario envía los datos y la página llama `$taller->modificarServicio($id, $desc, $costo)`; Taller hace el UPDATE

5. Encapsulamiento a nivel de sistema

- **Contraseñas:** se guardan con `password_hash()` y se comparan con `password_verify()`. La clase Usuario ni siquiera tiene un atributo de contraseña: el dato más sensible no sale nunca de la base.
- **Credenciales de la base:** viven en `inc/config.php`, archivo fuera del repositorio (`.gitignore`) y bloqueado por Apache (HTTP 403). El código público no contiene secretos.
- **Sesión:** solo se guarda el id del usuario; cada página reconstruye el objeto con `usuarioActual()` y `exigirRol()` decide si puede entrar. La página no decide, el objeto Usuario responde con `esAdmin()`.
- **Salida HTML:** todo texto que viene del usuario pasa por `htmlspecialchars()` al mostrarse.

6. Lo que se gana con esto

Sin encapsulamiento	Con encapsulamiento (como está el proyecto)
`$s->costoBase = "hola";`	**PHP lo impide: el atributo es private. Solo `setCostoBase()` puede cambiarlo.**
Cada página con su propio SQL	**Solo Taller conoce las tablas. Cambiar una columna se arregla en un archivo.**
Un cliente cambia su rol	**Usuario no tiene setRol(); el rol solo lo pone la base de datos.**
Validar en 10 lugares	**Validar en el setter o en el método de Taller, una sola vez.**

Resumen para la revisión: 11 clases, 0 atributos públicos, getters y setters con `$this` en todas, setters omitidos a propósito donde el dato no debe cambiar, y la conexión a la base de datos privada y sin getter dentro de Taller.